

p0448r4 - A stringstream replacement using `span<charT>` as buffer

Peter Sommerlad (peter.cpp@sommerlad.ch)

2021-02-26

Document Number:	p0448r4 (N2065 done right?)
Date:	2021-02-26
Project:	Programming Language C++
Audience:	LEWG/LWG

1 History

Streams have been the oldest part of the C++ standard library and especially `stringstreams` that can use pre-allocated buffers have been deprecated for a long time now, waiting for a replacement. p0407 and p0408 provide the efficient access to the underlying buffer for `stringstreams` that `stringstream` provided solving half of the problem that `stringstreams` provide a solution for. The other half is using a fixed size pre-allocated buffer, e.g., allocated on the stack, that is used as the stream buffers internal storage.

A combination of external-fixed and internal-growing buffer allocation that `stringstreambuf` provides is IMHO a doomed approach and very hard to use right.

There had been a proposal for the pre-allocated external memory buffer streams in N2065 but that went nowhere. Today, with `span<T>` we actually have a library type representing such buffers views we can use for specifying (and implementing) such streams. They can be used in areas where dynamic (re-)allocation of `stringstreams` is not acceptable but the burden of caring for a pre-existing buffer during the lifetime of the stream is manageable.

1.1 Changes from p0448r3

Incorporated changes as of LWG zoom 2021-02-19.

Changed the changes proposed in section 6.4 and forwarded by LEWG according to LWG feedback:

- Put all constraints on ROS in *Constraints*: especially ROS models `ranges::borrowed_range`
- explicitly convert to `std::span<const charT>` of ROS parameter.
- use `span::data` `span::size` member functions.
- removed `:` after Equivalent to where only an expression follows.

1.2 Changes from p0448r2

Based on the review of p0408 in Cologne 2019, some adjustments were made here accordingly to the feedback given there.

- update editorial advise to include header and feature test macro, as well as the section number to 29.
- made wording adjustments like suggested for p0408 in Cologne 2019 by LWG in the hope to make reviewing it faster.
- removed mentioning a template parameter `Extent` in all uses of `span`.
- fix the adopted wording (from `basic_stringbuf`) wrt seekoff by removing tables in favor of words.
- removed impossible to achieve `noexcept` from move and swap operations.
- some more email and review feedback from LWG telecon (spelling, spacing, etc).
- rephrased constructor wording to read more modern (Initializes vs constructs).
- added author email.
- fixed uses of correct macros to guarantee "Preconditions" and "Postconditions" wording vs. requires/ensures.
- added potential future change (not in the wording part) to provide `ispanstream` for `charT const*` arguments (this should be put in, before the next release, it is a kind of design bug fix.).

1.3 Changes from p0448r1

There was email discussion (Alisdair, Marshall, Titus and library mailing list) on semantics of move, timing and wording of `strstream` removal. Therefore, this paper needs to be reconsidered with that design respect by LEWG. I also acquired an additional paper number for a paper to propose the `strstream` removal, so I drop it from here.

Marshall gave a list of review comments, I'd like to answer below:

- The synopsis shows these classes in `std::experimental`, while the class descriptions show `std::` only. *fixed, copy relict.*
- The synopsis should probably `#include <span>` and `<string>`, since that's where `span` and `char_traits` come from. *yes to not to <string> since the base class `basic_streambuf` already has a dependency to `char_traits`, so no gain from mentioning <string>, but including <streambuf> might be shown. Fixed. However, I found no precedence to such include directives for stream classes in n4791 (may be a more modern style of specification introduced with C++11. I guess mentioning a required identifier encourages implementors to make its definition available.*
- Why a separate `<spanstream>` header? why not just put it in one of the existing ones? (we're adding headers at a surprising - to me - rate) *First, because `strstreams` are also in their separate header. Second, LEWG blessed/asked for it. Third, the base class already has the dependency to `char_traits`.*

- 7.4.2/1 is really generic: "Move assigns the base and members of *this from the base and corresponding members of rhs." *These words are almost identical to basic_istream move assignment. Took the challenge and now use (more) code.*
- 7.4.2/2 is mixing prose and code; I suspect it would be better just as code. "Effects Equivalent to: <two lines of code>" *almost identical to basic_istream::swap wording. see above.*
- Is the span that you pass to the constructors required to be non-empty? setbuf does have that requirement. *The latter is not really true: setbuf() is defined per streambuf subclass and we are free to define it any way. most subclasses say that setbuf(0,0) has no effect, filebuf makes I/O unbuffered and all say any other combination has implementation defined behavior. I do not require a non-empty span, the stream is then just not particularly useful, except to behave as a null object.*

Alisdair raised the question if the spanbuf move operations should actually disassociate the buffer-/stream from the original span, like (all?) other streambuf subclasses to when moved from.

"I have a huge concern about the definition of move construction and move assignment for basic_spanbuf. The reason is that this is simply a copy operation, but we allowed move semantics on streams/buffers following the unique ownership principle. In other words, it would be very surprising that writing to the move-from stream would have any impact on the moved-to stream."

Titus had the counter argument that one should not spend cycles on cleaning up moved from objects.

The streambuf base class can only be copied. filebuf and stringbuf both disassociate the right hand side from its underlying data source that they both own. strstreambuf does neither support move or copy.

I am torn, so I made that implementation defined.

Now to what really changed...

- rebase to n4791
- removed superfluous experimental namespace from synopsis
- added header includes in header synopsis for <streambuf> and (even so no other istream headers seem to do so).
- introduce an exposition-only member **span<charT> buf** representing the span. This will make wording, especially of move constructor more clear.
- make the wording of the move constructor more clear instead of hand waving about "locale and other state of rhs".
- make wording of spanbuf/streams's members more clear by code instead of weasel wording obtained from stringbuf/streams.
- TODO

1.4 Changes from p0448r0

- provide explanation why non-copy-ability, while technically feasible, is an OK thing.
- remove wrong Allocator template parameter (we never allocate anything).
- adhere to new section numbering of the standard.
- tried to clarify lifetime and threading issues.

2 Introduction

This paper proposes a class template `basic_spanbuf` and the corresponding stream class templates to enable the use of streams on externally provided memory buffers. No ownership or re-allocation support is given. For those features we have string-based streams.

3 Acknowledgements

- Thanks to those ISO C++ meeting members attending the Oulu meeting encouraging me to write this proposal. I believe Neil and Pablo have been among them, but can't remember who else.
- Thanks go to Jonathan Wakely who pointed the problem of `stringstream` out to me and to Neil Macintosh to provide the span library type specification.
- Thanks to Felix Morgner for proofreading.
- Thanks to Kona LEWG small group discussion suggesting some clarifications and Thomas Köppe for allowing me to use using type aliases instead of `typedef`.
- Thanks to remote LWG meeting December 2020 and surrounding email feedback by Jeff Garland, Tim Song, Jens Maurer, Volle Voutilainen. Special thanks to Tim for pointing out the `const charT*` potential for `basic_istream`.

4 Motivation

To finally get rid of the deprecated `stringstream` in the C++ standard we need a replacement. p0407/p0408 provide one for one half of the needs for `stringstream`. This paper provides one for the second half: fixed sized buffers.

[*Example*: reading input from a fixed pre-arranged character buffer:

```
char input[] = "10 20 30";
istream is{span<char>{input}};
int i;
is >> i;
ASSERT_EQUAL(10,i);
is >> i;
ASSERT_EQUAL(20,i);
is >> i;
```

```

ASSERT_EQUAL(30,i);
is >>i;
ASSERT(!is);

```

— *end example*] [*Example: writing to a fixed pre-arranged character buffer:*

```

char  output[30]{}; // zero-initialize array
ostream os{span<char>{output}};
os << 10 << 20 << 30;
auto const sp = os.span();
ASSERT_EQUAL(6,sp.size());
ASSERT_EQUAL("102030",std::string(sp.data(),sp.size()));
ASSERT_EQUAL(static_cast<void*>(output),sp.data()); // no copying of underlying data!
ASSERT_EQUAL("102030",output); // initialization guaranteed NUL termination

```

— *end example*]

5 Impact on the Standard

This is an extension to the standard library to enable deletion of the deprecated `strstream` classes by providing `basic_spanbuf`, `basic_spanstream`, `basic_istream`, and `basic_ostream` class templates that take an object of type `span<charT>` which provides an external buffer to be used by the stream.

It also proposes to remove the deprecated `strstreams` [`depr.str.strstreams`] assuming p0407 is also included in the standard.

6 Design Decisions

6.1 General Principles

The design follows from the principles of the `iostream` library. If discussed a person knowledgeable about `iostream`'s implementation is favorable, because of its many legacy design decisions, that would no longer be taken by modern C++ class designers. The behavior presented is part of what "frozen" `strstreams` provide, namely relying on a pre-allocated buffer, without the idiosyncrasy of `(o)strstream` that automatically (re-)allocates a new buffer on the C-heap, when the original buffer is insufficient for the output, which happens when such a buffer is not explicitly marked as "frozen". This broken design is the reason it has long been deprecated, but its use with pre-allocated buffers is one of the reasons it has not been banned completely, yet. Together with p0407 this paper gets rid of it.

As with all existing stream classes, using a stream object or a `streambuf` object from multiple threads can result in a data race. Only the pre-defined global stream objects `cin/cout/cerr` are exempt from this.

6.2 Older Open Issues (to be) Discussed by LEWG / IWG

- Should arbitrary types as template arguments to `span` be allowed to provide the underlying buffer by using the `byte` sequence representation `span` provides. (I do not think so and some

people in LEWG unofficially agree with it). You can always get a span of characters from the underlying byte sequence, so there is no need to put that functionality into spanbuf, it would break orthogonality and could lead to undefined behavior, because the streambuf would be aliasing with an arbitrary object.

- Should the `basic_spanbuf` be copy-able? It doesn't own any resources, so copying like with handles or `span` might be fine. Other concrete streambuf classes in the standard that own their buffer (`basic_stringbuf`, `basic_filebuf`) naturally prohibit copying, where the base class `basic_streambuf` provides a protected copy-ctor. I considered providing copyability for `basic_spanbuf`, because the implementation is `=default`. Note, none of the stream classes in the standard is copyable as are the stream classes provided here. Other streambuf subclasses are not copyable, mainly because they either represent an external resource (`fstreambuf`), or because one usually would not access it via its concrete type and only through its `basic_streambuf` abstraction, i.e., by using an associated stream's `rdbuf()` member function. I speculate that another reason, why `basic_stringbuf` is not copyable, is that copying its underlying string and re-establishing a new stream with it is possible and copying a streambuf felt not natural. Therefore, I stick with my decision to prohibit copying `basic_spanbuf`.

6.3 Current (r2) Open Issues (to be) Discussed by LEWG / LWG

- Should we keep a separate header `<spanstream>` ? Where to put it instead? [LEWG\(Kona2019\): yes!](#)
- Is adding a default constructor for `basic_spanbuf` OK? [LEWG\(Kona2019\): yes!](#)
- [LEWG\(Kona2019\): Forward to LWG for C++20!](#)

6.4 Future Directions (after p0448 is accepted for the WP: support `span<const charT>`)

During the finalizing review sessions Tim Song recognized a missing feature that `istream` supports, namely the use of the stream with a read-only input sequence (`char const*` in the case of `istream`). Some thinking it turned out that with a few additions and one change to class template `basic_istream` this could be achieved, because an input only `basic_spanbuf` will never attempt to modify its underlying character sequence, because `ebackfail()` is not overridden.

With Tim Song's help the following extra constructor from a read-only `span<charT const>` was provided.

~~The following changes will be proposed either by a follow-up paper or if LEWG gives immediate blessing~~ At the LEWG zoom 20210216 the change was supported unanimously. The changes marked below are applied to section 7 of this paper revision. The change delta is kept here for LWG's convenience to see what needs to be re-reviewed.:

LWG question: should the requirement go as code or as text? Mailing list reply was to put it as text, so the changes to section 7 reflect that and the changes marked below show it.

Change the synopsis of `[istream]` as follows:

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_istream
```

```

    : public basic_istream<charT, traits> {
public:
    using char_type      = charT;
    using int_type       = typename traits::int_type;
    using pos_type       = typename traits::pos_type;
    using off_type       = typename traits::off_type;
    using traits_type     = traits;

    // [ispanstream.cons], constructors
    explicit basic_ispanstream(
        std::span<charT> s,
        ios_base::openmode which = ios_base::in);

    template<class ROS>
    explicit basic_ispanstream(ROS&& s);

    basic_ispanstream(const basic_ispanstream&) = delete;
    basic_ispanstream(basic_ispanstream&& rhs);

    // [ispanstream.swap], assign and swap
    basic_ispanstream& operator=(const basic_ispanstream&) = delete;
    basic_ispanstream& operator=(basic_ispanstream&& rhs);
    void swap(basic_ispanstream& rhs);

    // [ispanstream.members], members
    basic_spanbuf<charT, traits>* rdbuf() const noexcept;

    std::span<const charT> span() const noexcept;
    void span(std::span<charT> s) noexcept;

    template<class ROS>
    void span(ROS&& s) noexcept ;

private:
    basic_spanbuf<charT, traits> sb; // exposition only
};

template<class charT, class traits>
    void swap(basic_ispanstream<charT, traits>& x,
              basic_ispanstream<charT, traits>& y);
}

```

[Note: Constructing an `ispanstream` from a *string-literal* will include the termination character `'\0'` in the underlying `spanbuf`. — end note]

Add to *[ispanstream.cons]* the following

```

template<class ROS>
explicit basic_ispanstream(ROS&& s)

```

¹ Constraints: `ROS` models `ranges::borrowed_range`.
 (!convertible_to<ROS, std::span<charT>>) &&
 convertible_to<ROS, std::span<const charT>> is true.

2 *Effects:* Let `sp` be `std::span<const charT>(std::forward<ROS>(s))`. Equivalent to `basic_ispanstream(std::span<charT>(const_cast<charT*>(sp.data()),sp.size()))`.

Change [ispanstream.members] as follows

6.4.1 29.x.3.3 Member functions [ispanstream.members]

```
basic_spanbuf<charT, traits>* rdbuf() const noexcept;
```

1 *Effects:* Equivalent to:
`return const_cast<basic_spanbuf<charT, traits>*>(addressof(sb));`

```
std::span<const charT> span() const noexcept;
```

2 *Effects:* Equivalent to:
`return rdbuf()->span();`

```
void span(std::span<charT> s) noexcept;
```

3 *Effects:* Equivalent to `rdbuf()->span(s)`.

```
template<class ROS>
void span(ROS&& s) noexcept;
```

4 *Constraints:* `ROS` models `ranges::borrowed_range`.
`(!convertible_to<ROS, std::span<charT>>) &&`
`convertible_to<ROS, std::span<const charT>>` is true.

5 *Effects:* Let `sp` be `std::span<const charT>(std::forward<ROS>(s))`. Equivalent to `this->span(std::span<const charT>(const_cast<charT*>(sp.data()),sp.size()))`.

7 Technical Specifications

Introduce a new header `<spanstream>` in subclause ([headers]): Table 19 ([tab:headers.cpp]).

In section [version.syn] add the feature test macro `__cpp_lib_spanstream` with the corresponding value for the header `<spanstream>`:

```
#define __cpp_lib_spanstream TBD // also in <spanstream>
```

Insert a new section 29.x in chapter 29 [input.output] after section 29.8 [string.streams] and adjust table [tab:iostreams.summary] accordingly.

7.1 29.3.1 Header `<iosfwd>` synopsis [iosfwd.syn]

In 29.3.1 [iosfwd.syn] add the following forward declarations and type aliases in the appropriate places.

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
        class basic_spanbuf;
    template<class charT, class traits = char_traits<charT>>
        class basic_ispanstream;
    template<class charT, class traits = char_traits<charT>>
        class basic_ospstream;
```



```

template<class charT, class traits = char_traits<charT>>
    class basic_spanstream;

using spanbuf      = basic_spanbuf<char>;
using ispanstream  = basic_ispanstream<char>;
using ospanstream  = basic_ostream<char>;
using spanstream   = basic_spanstream<char>;

using wspanbuf     = basic_spanbuf<wchar_t>;
using wispanstream = basic_ispanstream<wchar_t>;
using wospanstream = basic_ostream<wchar_t>;
using wspanstream  = basic_spanstream<wchar_t>;
}

```

7.2 29.x Span-based Streams [span.streams]

- ¹ This section introduces a stream interface for user-provided fixed-size buffers.

7.2.1 29.x.1 Overview [span.streams.overview]

- ¹ The header `<spanstream>` defines class templates and types that associate stream buffers with objects whose types are specializations of `span` as described in [views.span].

[*Note:* A user of these classes is responsible for ensuring that the character sequence represented by the given `span` outlives the use of the sequence by objects of the classes in this subclause. Using multiple `basic_spanbuf` objects referring to overlapping underlying sequences from different threads, where at least one `basic_spanbuf` object is used for writing to the sequence, results in a data race. — *end note*]

7.2.2 Header `<spanstream>` synopsis [span.streams.syn]

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
        class basic_spanbuf;

    using spanbuf = basic_spanbuf<char>;
    using wspanbuf = basic_spanbuf<wchar_t>;

    template<class charT, class traits = char_traits<charT>>
        class basic_ispanstream;

    using ispanstream = basic_ispanstream<char>;
    using wispanstream = basic_ispanstream<wchar_t>;

    template<class charT, class traits = char_traits<charT>>
        class basic_ostream;

    using ostream = basic_ostream<char>;
    using wostream = basic_ostream<wchar_t>;

    template<class charT, class traits = char_traits<charT>>
        class basic_spanstream;
}

```

```

using spanstream = basic_spanstream<char>;
using wspanstream = basic_spanstream<wchar_t>;
}

```

7.3 29.x.2 Class template basic_spanbuf [spanbuf]

```

namespace std {
    template<class charT, class traits = char_traits<charT> >
    class basic_spanbuf
        : public basic_streambuf<charT, traits> {
    public:
        using char_type      = charT;
        using int_type        = typename traits::int_type;
        using pos_type        = typename traits::pos_type;
        using off_type        = typename traits::off_type;
        using traits_type     = traits;

        // [spanbuf.cons], constructors
        basic_spanbuf() : basic_spanbuf(ios_base::in | ios_base::out) {}
        explicit basic_spanbuf(ios_base::openmode which)
            : basic_spanbuf(std::span<charT>(), which) {}
        explicit basic_spanbuf(
            std::span<charT> s,
            ios_base::openmode which = ios_base::in | ios_base::out);
        basic_spanbuf(const basic_spanbuf&) = delete;
        basic_spanbuf(basic_spanbuf&& rhs);

        // [spanbuf.assign], assign and swap
        basic_spanbuf& operator=(const basic_spanbuf&) = delete;
        basic_spanbuf& operator=(basic_spanbuf&& rhs);
        void swap(basic_spanbuf& rhs);

        // [spanbuf.members], get and set
        std::span<charT> span() const noexcept;
        void span(std::span<charT> s) noexcept;

    protected:
        // [spanbuf.virtuals], overridden virtual functions
        basic_streambuf<charT, traits>* setbuf(charT*, streamsize) override;

        pos_type seekoff(off_type off, ios_base::seekdir way,
            ios_base::openmode which
            = ios_base::in | ios_base::out) override;
        pos_type seekpos(pos_type sp,
            ios_base::openmode which
            = ios_base::in | ios_base::out) override;

    private:
        ios_base::openmode mode; // exposition only
        std::span<charT> buf; // exposition only
    };
}

```

```

    template<class charT, class traits>
        void swap(basic_spanbuf<charT, traits>& x,
                  basic_spanbuf<charT, traits>& y);
}

```

¹ The class template `basic_spanbuf` is derived from `basic_streambuf` to associate possibly the input sequence and possibly the output sequence with a sequence of arbitrary characters. The sequence is provided by an object of class `span<charT>`.

² For the sake of exposition, the maintained data is presented here as:

(2.1) — `ios_base::openmode mode`, has `in` set if the input sequence can be read, and `out` set if the output sequence can be written.

(2.2) — `std::span<charT> buf` is the view to the underlying character sequence.

7.4 29.x.2.1 `basic_spanbuf` constructors [spanbuf.cons]

```

explicit basic_spanbuf(
    std::span<charT> s,
    ios_base::openmode which = ios_base::in | ios_base::out);

```

¹ *Effects:* Initializes the base class with `basic_streambuf()` ([streambuf.cons]), and `mode` with `which`. Initializes the internal pointers as if calling `span(s)`.

```

basic_spanbuf(basic_spanbuf&& rhs);

```

² *Effects:* Initializes the base class with `std::move(rhs)` and `mode` with `std::move(rhs.mode)` and `buf` with `std::move(rhs.buf)`. The sequence pointers in `*this` (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) obtain the values which `rhs` had. It is implementation-defined whether `rhs.buf.empty()` returns true after the move.

³ *Postconditions:* Let `rhs_p` refer to the state of `rhs` just prior to this construction.

(3.1) — `span().data() == rhs_p.span().data()`

(3.2) — `span().size() == rhs_p.span().size()`

(3.3) — `eback() == rhs_p.eback()`

(3.4) — `gptr() == rhs_p.gptr()`

(3.5) — `egptr() == rhs_p.egptr()`

(3.6) — `pbase() == rhs_p.pbase()`

(3.7) — `pptr() == rhs_p.pptr()`

(3.8) — `epptr() == rhs_p.epptr()`

(3.9) — `getloc() == rhs_p.getloc()`

7.4.1 29.x.2.2 Assign and swap [spanbuf.assign]

```

basic_spanbuf& operator=(basic_spanbuf&& rhs);

```

¹ *Effects:* Equivalent to:

```

    basic_spanbuf tmp{std::move(rhs)};

```

```

        this->swap(tmp);
        return *this;

```

```

void swap(basic_spanbuf& rhs);

```

2 *Effects:* Equivalent to:

```

        basic_streambuf<charT, traits>::swap(rhs);
        std::swap(mode, rhs.mode);
        std::swap(buf, rhs.buf);

```

```

template<class charT, class traits>
    void swap(basic_spanbuf<charT, traits>& x,
              basic_spanbuf<charT, traits>& y);

```

3 *Effects:* Equivalent to `x.swap(y)`.

7.4.2 29.x.2.3 Member functions [spanbuf.members]

```

std::span<charT> span() const;

```

1 *Returns:* If `ios_base::out` is set in mode, returns `std::span<charT>(pbase(), pptr())`, otherwise returns `buf`.

[*Note:* In contrast to `basic_stringbuf`, the underlying sequence never grows and is not owned. An owning copy can be obtained by converting the result to `basic_string<charT>`. — *end note*]

```

void span(std::span<charT> s);

```

2 *Effects:* `buf = s`. Initializes the input and output sequences according to mode.

3 *Postconditions:*

- (3.1) — If `ios_base::out` is set in mode, `(pbase() == s.data() && eptr() == pbase() + s.size())` is true;
- (3.1.1) — in addition, if `ios_base::ate` is set in mode, `pptr() == pbase() + s.size()` is true,
- (3.1.2) — otherwise `pptr() == pbase()` is true.
- (3.2) — If `ios_base::in` is set in mode, `(eback() == s.data() && gptr() == eback() && egptr() == eback() + s.size())` is true.

7.4.3 29.x.2.4 Overridden virtual functions [spanbuf.virtuals]

1 [*Note:* Because the underlying buffer is of fixed size, neither overflow, underflow, nor `pbackfail` can provide useful behavior. — *end note*]

```

pos_type seekoff(off_type off, ios_base::seekdir way,
                 ios_base::openmode which
                 = ios_base::in | ios_base::out) override;

```

2 *Effects:* Alters the stream position within one or both of the controlled sequences, if possible, as follows:

- (2.1) — if `ios_base::in` is set in `which`, positions the input sequence; `xnext` is `gptr()`, `xbeg` is `eback()`.
- (2.2) — if `ios_base::out` is set in `which`, positions the output sequence; `xnext` is `pptr()`, `xbeg` is `pbase()`.
- 3 If both `ios_base::in` and `ios_base::out` are set in `which` and `way` is `ios_base::cur` the positioning operation fails.
- 4 For a sequence to be positioned, if its next pointer `xnext` (either `gptr()` or `pptr()`) is a null pointer and the new offset `newoff` as computed below is nonzero, the positioning operation fails. Otherwise, the function determines `baseoff` as a value of type `off_type` as follows:
- (4.1) — 0 when `way` is `ios_base::beg`;
- (4.2) — `(pptr() - pbase())` for the output sequence, or
`(gptr() - eback())` for the input sequence when `way` is `ios_base::cur`;
- (4.3) — when `way` is `ios_base::end` :
- (4.3.1) — `(pptr() - pbase())` if `ios_base::out` is set in mode and `ios_base::in` is not set in mode,
- (4.3.2) — `buf.size()` otherwise.
- 5 If `baseoff + off` would overflow, or if `baseoff + off` is less than zero, or if `baseoff + off` is greater than `buf.size()`, the positioning operation fails. Otherwise, the function computes
`off_type newoff = baseoff + off;`
and assigns `xbeg + newoff` to the next pointer `xnext`.
- 6 *Returns:* `pos_type(off_type(-1))` if the positioning operation fails; `pos_type(newoff)` otherwise.
- ```
pos_type seekpos(pos_type sp,
 ios_base::openmode which
 = ios_base::in | ios_base::out) override;
```
- 7 *Effects:* Equivalent to:  
`return seekoff(off_type(sp), ios_base::beg, which);`
- ```
basic_streambuf<charT, traits>* setbuf(charT* s, streamsize n);
```
- 8 *Effects:* Equivalent to `this->span(std::span<charT>(s, n)).`
- 9 *Returns:* `this`.

7.5 29.x.3 Class template `basic_ispanstream` [`ispanstream`]

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ispanstream
    : public basic_istream<charT, traits> {
    public:
        using char_type      = charT;
        using int_type       = typename traits::int_type;
```

```

using pos_type      = typename traits::pos_type;
using off_type      = typename traits::off_type;
using traits_type    = traits;

// [ispanstream.cons], constructors
explicit basic_ispanstream(
    std::span<charT> s,
    ios_base::openmode which = ios_base::in);
basic_ispanstream(const basic_ispanstream&) = delete;
basic_ispanstream(basic_ispanstream&& rhs);
template<class ROS>
explicit basic_ispanstream(ROS&& s);

// [ispanstream.swap], assign and swap
basic_ispanstream& operator=(const basic_ispanstream&) = delete;
basic_ispanstream& operator=(basic_ispanstream&& rhs);
void swap(basic_ispanstream& rhs);

// [ispanstream.members], members
basic_spanbuf<charT, traits>* rdbuf() const noexcept;

std::span<const charT> span() const noexcept;
void span(std::span<charT> s) noexcept;
template<class ROS>
void span(ROS&& s) noexcept;

private:
    basic_spanbuf<charT, traits> sb; // exposition only
};

template<class charT, class traits>
void swap(basic_ispanstream<charT, traits>& x,
          basic_ispanstream<charT, traits>& y);
}

```

¹ [Note: Constructing an `ispanstream` from a *string-literal* will include the termination character `'\0'` in the underlying `spanbuf`. — end note]

7.5.1 29.x.3.1 `basic_ispanstream` constructors [ispanstream.cons]

```

explicit basic_ispanstream(
    std::span<charT> s,
    ios_base::openmode which = ios_base::in);

```

¹ *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` and `sb` with `basic_spanbuf<charT, traits>(s, which | ios_base::in)` ([spanbuf.cons]).

```

basic_ispanstream(basic_ispanstream&& rhs);

```

² *Effects:* Initializes the base class with `std::move(rhs)` and `sb` with `std::move(rhs.sb)`. Next, `basic_istream<charT, traits>::set_rdbuf(addressof(sb))` is called to install the contained `basic_spanbuf`.

```

template<class ROS>
explicit basic_ispanstream(ROS&& s)
3   Constraints: ROS models ranges::borrowed_range.
    (!convertible_to<ROS, std::span<charT>>) &&
    convertible_to<ROS, std::span<charT const>> is true.
4   Effects: Let sp be std::span<const charT>(std::forward<ROS>(s)). Equivalent to
    basic_ispanstream(std::span<charT>(const_cast<charT*>(sp.data()), sp.size())).

```

7.5.2 29.x.3.2 Swap [ispanstream.swap]

```

void swap(basic_ispanstream& rhs);
1   Effects: Equivalent to:
        basic_istream<charT, traits>::swap(rhs);
        sb.swap(rhs.sb);

```

```

template<class charT, class traits>
void swap(basic_ispanstream<charT, traits>& x,
        basic_ispanstream<charT, traits>& y);

```

2 *Effects:* Equivalent to x.swap(y).

7.5.3 29.x.3.3 Member functions [ispanstream.members]

```

basic_spanbuf<charT, traits>* rdbuf() const noexcept;

```

1 *Effects:* Equivalent to:
return const_cast<basic_spanbuf<charT, traits>*>(addressof(sb));

```

std::span<const charT> span() const noexcept;

```

2 *Effects:* Equivalent to:
return rdbuf()->span();

```

void span(std::span<charT> s) noexcept;

```

3 *Effects:* Equivalent to rdbuf()->span(s).

```

template<class ROS>
void span(ROS&& s) noexcept;

```

4 *Constraints:* ROS models ranges::borrowed_range.
(!convertible_to<ROS, std::span<charT>>) &&
convertible_to<ROS, std::span<charT const>> is true.

5 *Effects:* Let sp be std::span<const charT>(std::forward<ROS>(s)). Equivalent to
this->span(std::span<charT>(const_cast<charT*>(sp.data()), sp.size())).

7.6 29.x.4 Class template basic_ostream [ostream]

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ostream

```

```

    : public basic_ostream<charT, traits> {
public:
    using char_type      = charT;
    using int_type       = typename traits::int_type;
    using pos_type       = typename traits::pos_type;
    using off_type       = typename traits::off_type;
    using traits_type     = traits;

    // [ospanstream.cons], constructors
    explicit basic_ospanstream(
        std::span<charT> s,
        ios_base::openmode which = ios_base::out);
    basic_ospanstream(const basic_ospanstream&) = delete;
    basic_ospanstream(basic_ospanstream&& rhs);

    // [ospanstream.swap], assign and swap
    basic_ospanstream& operator=(const basic_ospanstream&) = delete;
    basic_ospanstream& operator=(basic_ospanstream&& rhs);
    void swap(basic_ospanstream& rhs);

    // [ospanstream.members], members
    basic_spanbuf<charT, traits>* rdbuf() const noexcept;

    std::span<charT> span() const noexcept;
    void span(std::span<charT> s) noexcept;
private:
    basic_spanbuf<charT, traits> sb; // exposition only
};

template<class charT, class traits>
    void swap(basic_ospanstream<charT, traits>& x,
              basic_ospanstream<charT, traits>& y);
}

```

7.6.1 29.x.4.1 basic_ospanstream constructors [ospanstream.cons]

```

explicit basic_ospanstream(
    std::span<charT> s,
    ios_base::openmode which = ios_base::out);

```

- 1 *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` and `sb` with `basic_spanbuf<charT, traits>(span, which | ios_base::out)` ([spanbuf.cons]).

```

basic_ospanstream(basic_ospanstream&& rhs) noexcept;

```

- 2 *Effects:* Initializes the base class with `std::move(rhs)` and `sb` with `std::move(rhs.sb)`. Next, `basic_ostream<charT, traits>::set_rdbuf(addressof(sb))` is called to install the contained `basic_spanbuf`.

7.6.2 29.x.4.2 Swap [ospanstream.swap]

```

void swap(basic_ospanstream& rhs);

```


1 *Effects:* Equivalent to:

```

        basic_ostream<charT, traits>::swap(rhs);
        sb.swap(rhs.sb);

```

```

template<class charT, class traits>
void swap(basic_ostream<charT, traits>& x,
        basic_ostream<charT, traits>& y);

```

2 *Effects:* Equivalent to `x.swap(y)`.

7.6.3 29.x.4.3 Member functions [ostream.members]

```

basic_ostreambuf<charT, traits>* rdbuf() const noexcept;

```

1 *Effects:* Equivalent to:

```

        return const_cast<basic_ostreambuf<charT, traits>*>(addressof(sb));

```

```

std::span<charT> span() const noexcept;

```

2 *Effects:* Equivalent to:

```

        return rdbuf()->span();

```

```

void span(std::span<charT> s) noexcept;

```

3 *Effects:* Equivalent to `rdbuf()->span(s)`.

7.7 29.x.5 Class template basic_ostream [ostream]

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ostream
        : public basic_istream<charT, traits> {
    public:
        using char_type      = charT;
        using int_type       = typename traits::int_type;
        using pos_type       = typename traits::pos_type;
        using off_type       = typename traits::off_type;
        using traits_type    = traits;

        // [ostream.cons], constructors
        explicit basic_ostream(
            std::span<charT> s,
            ios_base::openmode which = ios_base::out | ios_base::in);
        basic_ostream(const basic_ostream&) = delete;
        basic_ostream(basic_ostream&& rhs);

        // [ostream.swap], assign and swap
        basic_ostream& operator=(const basic_ostream&) = delete;
        basic_ostream& operator=(basic_ostream&& rhs);
        void swap(basic_ostream& rhs);

        // [ostream.members], members
        basic_ostreambuf<charT, traits>* rdbuf() const noexcept;

```

```

    std::span<charT> span() const noexcept;
    void span(std::span<charT> s) noexcept;
private:
    basic_spanbuf<charT, traits> sb; // exposition only
};

template<class charT, class traits>
    void swap(basic_spanstream<charT, traits>& x,
              basic_spanstream<charT, traits>& y);
}

```

7.7.1 29.x.5.1 basic_spanstream constructors [spanstream.cons]

```

explicit basic_spanstream(
    std::span<charT> s,
    ios_base::openmode which = ios_base::out | ios_base::in);

```

- 1 *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` and `sb` with `basic_spanbuf<charT, traits>(s, which)` ([spanbuf.cons]).

```

basic_spanstream(basic_spanstream&& rhs);

```

- 2 *Effects:* Initializes the base class with `std::move(rhs)` and `sb` with `std::move(rhs.sb)`. Next, `basic_iostream<charT, traits>::set_rdbuf(addressof(sb))` is called to install the contained `basic_spanbuf`.

7.7.2 29.x.5.2 Swap [spanstream.swap]

```

void swap(basic_spanstream& rhs);

```

- 1 *Effects:* Equivalent to:
- ```

 basic_iostream<charT, traits>::swap(rhs);
 sb.swap(rhs.sb);

```

```

template<class charT, class traits>
 void swap(basic_spanstream<charT, traits>& x,
 basic_spanstream<charT, traits>& y);

```

- 2     *Effects:* Equivalent to `x.swap(y)`.

### 7.7.3 29.x.5.3 Member functions [spanstream.members]

```

basic_spanbuf<charT, traits>* rdbuf() const noexcept;

```

- 1     *Effects:* Equivalent to:
- ```

    return const_cast<basic_spanbuf<charT, traits>*>(addressof(sb));

```

```

std::span<charT> span() const noexcept;

```

- 2 *Effects:* Equivalent to:
- ```

 return rdbuf()->span();

```

```

void span(std::span<charT> s) noexcept;

```

- 3     *Effects:* Equivalent to `rdbuf()->span(s)`.

## 8 Appendix: Example Implementations

An example implementation is available under the author's github account at: [https://github.com/PeterSommerlad/SC22WG21\\_Papers/tree/master/workspace/p0448](https://github.com/PeterSommerlad/SC22WG21_Papers/tree/master/workspace/p0448)